SLAM Drone Project Handoff

SLAM-Based Autonomous Drone for GPS-Denied Indoor Navigation — Project Handoff

Team: Kishan Ballal, Yathivandith JP (ECE, B section) **Deadline:** 21 November 2026 **Status as of 3 November 2026:** Milestone 0 complete. Milestone 1 (first SLAM map) is next.

---

0. Instructions for the assistant continuing this project

- Give hands-on, step-by-step guidance with **exact commands** and **complete files**. Kishan is learning ROS 2 as he builds.
- Always say **which terminal tab** a command goes in (see Section 5). Kishan usually sends phone photos of his screen; read them carefully.
- Keep scope locked to the milestones in Section 8. Flag anything that risks the 21 Nov deadline.
- When something breaks, debug it before proposing a different approach.
- Each milestone must be demoable on its own: freeze it (backup/git tag + screen recording) before starting the next.
- **Do not add cameras** to the drone without first solving the WSL camera memory leak (Section 6).

---

1. Project goal

Problem: Drones fail indoors because they depend on GPS and a human pilot. Disaster sites, warehouses and industrial floors have no GPS, and piloting a drone there is impractical.

Solution: A simulated drone enters an unknown indoor building and maps it on its own: no GPS, no prebuilt map, no human control. It uses LiDAR SLAM for mapping and localisation, and frontier exploration to keep seeking unexplored areas until the whole building is covered. Final story: a search-and-rescue drone that maps a building and marks where victims are.

It must run in real time and look impressive in a demo video.

---

2. Hardware and environment

Item	Value
Laptop	HP Omen (HyperX), Intel Core i7, 24 GB RAM
GPU	NVIDIA GeForce RTX 5060 Laptop, 8 GB VRAM, Windows driver 621.01
Host OS	Windows 11 + WSL 3.0.3 (no dual boot, by choice)
Linux	Ubuntu 24.04 LTS inside WSL2, user `kisha`, host `HpHyperXomen`
ROS 2	Jazzy (`/opt/ros/jazzy`, sourced in `~/.bashrc`)
Simulator	Gazebo Harmonic (`gz sim`)
Flight stack	PX4-Autopilot **v1.17.0** SITL in `~/PX4-Autopilot`
PX4 ↔ ROS 2	Micro-XRCE-DDS-Agent v2.4.3 (built in `~/Micro-XRCE-DDS-Agent`, installed system-wide)
ROS workspace	`~/ws_px4` with `px4_msgs` (branch `release/1.17`) + `px4_ros_com` (main); sourced in `~/.bashrc`
Visualisation	RViz2, config `~/drone.rviz`

Performance: Gazebo runs at Real Time Factor ≈ 99–100% with the GPU.

Environment settings already applied

- `~/.bashrc` forces GPU rendering in WSL (without these, Gazebo ran on the CPU via `llvmpipe` and was very slow):

```
export GALLIUM_DRIVER=d3d12
export MESA_D3D12_DEFAULT_ADAPTER_NAME=NVIDIA
source /opt/ros/jazzy/setup.bash
source ~/ws_px4/install/setup.bash
```

- `C:\Users\kisha\.wslconfig` (caps WSL so a leak can't freeze Windows):

```
[wsl2]
memory=12GB
swap=8GB

[experimental]
autoMemoryReclaim=gradual
```

---

3. Custom files built (Milestone 0)

All are backed up in `~/slam_drone/m0/`.

3.1 Drone model `x500_lidar_3d` (the main project drone)

Path: `~/PX4-Autopilot/Tools/simulation/gz/models/x500_lidar_3d/model.sdf`

- A copy of PX4's `x500_lidar_2d` (keeps the stock 2D LiDAR `lidar_2d_v2`, 360°, mounted ~0.26 m above base).
- Added a **3D LiDAR** link `lidar_3d_link` at z = 0.32 m: `gpu_lidar`, Gazebo topic `lidar_3d` (points on `/lidar_3d/points`), 10 Hz, 900 horizontal × 16 vertical samples, vertical FOV ±15°, range 0.2–12 m (reduced from 30 m), Gaussian noise σ = 0.01.
- Memory is stable at ~2.7 GB after 5+ minutes. **No leak.**

3.2 Airframe `4098_gz_x500_lidar_3d`

Path: `~/PX4-Autopilot/ROMFS/px4fmu_common/init.d-posix/airframes/4098_gz_x500_lidar_3d` (registered in that folder's `CMakeLists.txt`). Built-in defaults appended:

```
param set-default NAV_DLL_ACT 0        # no "No connection to the GCS" block
param set-default COM_RC_IN_MODE 4     # no RC transmitter required
param set-default MIS_TAKEOFF_ALT 1.5  # walls are 3 m tall
```

Launch: `PX4_GZ_WORLD=building make px4_sitl gz_x500_lidar_3d`

3.3 World `building.sdf`

Path: `~/PX4-Autopilot/Tools/simulation/gz/worlds/building.sdf` (copy of PX4 `default.sdf` with a static `building` model added; world name `building`). Walls 3 m tall, 0.2 m thick. Coordinates are Gazebo world ENU (x east, y north); the drone spawns at the origin.

Element	Position / size
Outer walls	20 × 16 m box: x −10…10, y −8…8
Central "island" block	centre (0, 3), 10 × 3 m → x −5…5, y 1.5…4.5 (full height)
Corridor loop	around the island: south y −1.5…1.5, north y 4.5…8, east x 5…10, west x −10…−5
Partition wall (corridor ↔ rooms)	along y = −1.5, doorways at x ≈ −6 (gap −6.6…−5.4) and x ≈ 5 (gap 4.4…5.6)
Divider between the two rooms	x = 0, from y −8 to −1.5
SW room / SE room	x −10…0 / x 0…10, y −8…−1.5
Pillars (full height, 0.5 × 0.5)	(7.5, 2) east corridor; (−7.5, 5) west corridor
Table (0.75 m tall)	(−5, −5) SW room
Shelf (1.8 m tall)	(6, −6.5) SE room
Crate (1.0 m tall)	(3, 6.5) north corridor (the drone at 1.5 m flies over it)

Suggested corridor-loop waypoints (ENU, z = 1.5 m; they clear the pillars and island by ≥ 1 m): (0, 0) → (6.2, 0) → (6.2, 6) → (−6.2, 6) → (−6.2, 0) → (0, 0). For PX4 setpoints convert to local NED: x_ned = y_enu, y_ned = x_enu, z_ned = −z_enu (so altitude 1.5 m → z = −1.5).

3.4 Older custom drone (not used)

`x500_lidar_depth` (airframe `4099_gz_x500_lidar_depth`): depth camera + 2D LiDAR. **Abandoned** because of the camera memory leak (Section 6).

3.5 RViz config `~/drone.rviz`

Fixed Frame `base_link`. PointCloud2 `/lidar_3d/points` (Flat Squares, 0.02 m, AxisColor on Z). LaserScan `/scan` (0.05 m, FlatColor white). (Ignore the empty folder `~/lidar.rviz`, an accidental leftover; remove it with `rmdir ~/lidar.rviz`.)

---

4. Topics and frames

Data	Gazebo topic	ROS 2 topic	Notes
3D LiDAR	`/lidar_3d/points`	`/lidar_3d/points` (PointCloud2)	frame id from `header.frame_id` (≈ `lidar_3d_link`)
2D LiDAR	long name ending in `/scan`	remapped to `/scan` (LaserScan)	bridge finds it automatically
PX4 data	n/a	`/fmu/out/...`, `/fmu/in/...` via XRCE-DDS	PX4 v1.17 adds version suffixes: e.g. `/fmu/out/vehicle_status_v1`, `/fmu/out/vehicle_local_position_v1`. Always check `ros2 topic list | grep fmu` for exact names

Current TF tree (temporary, Milestone 0): static transforms only, `base_link → <3D lidar frame>` (z 0.32) and `base_link → <2D lidar frame>` (x 0.12, z 0.26). There is **no `odom` or `map` frame yet**; Milestone 1 adds them.

---

5. Session startup routine (4 Ubuntu tabs)

Open Ubuntu 24.04 from the Start menu; new tabs via ⌄ → **Ubuntu 24.04**. One running program per tab.

Tab 1 — simulation

```
pkill -f "gz sim"; pkill -f px4
cd ~/PX4-Autopilot
PX4_GZ_WORLD=building make px4_sitl gz_x500_lidar_3d
```

Wait for “Ready for takeoff!”. Fly with `commander takeoff` / `commander land` at the `pxh>` prompt (type each once).

Tab 2 — Gazebo → ROS 2 LiDAR bridge

```
SCAN=$(gz topic -l | grep "/scan$" | head -1)
echo "2D scan topic: $SCAN"
ros2 run ros_gz_bridge parameter_bridge \
  /lidar_3d/points@sensor_msgs/msg/PointCloud2[gz.msgs.PointCloudPacked \
  "$SCAN@sensor_msgs/msg/LaserScan[gz.msgs.LaserScan" \
  --ros-args -r "$SCAN:=/scan"
```

Tab 3 — RViz

```
rviz2 -d ~/drone.rviz
```

Tab 4 — sensor transforms (temporary)

```
F2D=$(ros2 topic echo /scan --field header.frame_id --once | head -1); echo "2D frame: $F2D"
F3D=$(ros2 topic echo /lidar_3d/points --field header.frame_id --once | head -1); echo "3D frame: $F3D"
ros2 run tf2_ros static_transform_publisher --x 0 --y 0 --z 0.32 --frame-id base_link --child-frame-id $F3D &
ros2 run tf2_ros static_transform_publisher --x 0.12 --y 0 --z 0.26 --frame-id base_link --child-frame-id $F2D &
```

PX4 ↔ ROS 2 bridge (when PX4 topics are needed), in its own tab:

```
MicroXRCEAgent udp4 -p 8888
```

Clean shutdown, in order: `commander land` → `shutdown` (Tab 1; Ctrl+C if the prompt doesn’t return) → Ctrl+C in the bridge tabs → close RViz (Ctrl+S first) → `pkill -f static_transform_publisher` → `pkill -f "gz sim"; pkill -f px4`. **If anything freezes:** close the tabs, run `wsl --shutdown` in Windows PowerShell, reopen Ubuntu. **Memory check:** `watch -n 2 free -h` (normal with the LiDAR drone: ~2–3 GB used, steady).

---

6. Known issues and fixes

Issue	Cause	Fix / status
Very slow sim (earlier attempt)	WSL rendering on CPU (`llvmpipe`)	d3d12 + NVIDIA adapter env vars in `.bashrc` → RTF ~100%

“Preflight Fail: No connection to the GCS”, arming denied	PX4 expects a ground station / RC	`NAV_DLL_ACT 0`, `COM_RC_IN_MODE 4` (built into airframe 4098)
Params lost after a crash	PX4 froze before saving	Now set as `param set-default` in the airframe file
`ros2 topic echo /fmu/out/vehicle_status` fails	PX4 v1.17 renamed topics	Use `_v1` suffixed names from `ros2 topic list`
`make` hangs after `shutdown`	Gazebo slow to exit	Ctrl+C, then `pkill -f "gz sim"; pkill -f px4`
All cameras leak memory (~100–150 MB/s) until WSL freezes	Gazebo camera rendering on WSL’s graphics stack. Tested: depth camera on GPU (leaks), depth on CPU/llvmpipe (leaks faster), mono colour camera (leaks), old `ogre` render engine (leaks). 2D and 3D LiDAR do **not** leak	**Cameras dropped**; 3D LiDAR used for 3D sensing. Untried fix: newer Mesa graphics library in Ubuntu
RViz “No tf data / frame does not exist”	Bridge not running, or no TF yet	Start the bridge; Fixed Frame `base_link`; static TFs in Tab 4
RViz huge blobs	Size (m) set to 3	Size 0.02–0.05 m

7. Completed so far

- **Day 1 (21 Oct):** NVIDIA driver, WSL2 + Ubuntu 24.04, GPU rendering fix, RAM cap, ROS 2 Jazzy + Gazebo Harmonic (shapes test RTF 99.92%), PX4 build, first takeoff/landing with x500.
- **Night of 21 Oct:** PX4 ↔ ROS 2 bridge (Micro-XRCE-DDS-Agent + px4_msgs); live `/fmu/out/*` data in ROS 2 confirmed.
- **22 Oct:** sensor tests (depth camera and 2D LiDAR drones); diagnosis of the camera memory leak; custom `x500_lidar_3d` drone (2D + 3D LiDAR) + airframe 4098; both LiDARs bridged into ROS 2 and shown in RViz; `building.sdf` world; drone hovering inside the building with walls visible live in RViz.
- ✅ **Milestone 0 complete:** drone with full LiDAR sensor suite in an indoor building, real time, live in ROS 2/RViz. Backed up to `~/slam_drone/m0`.
- 26 Oct – 1 Nov: no work (internal exams).
- A Day 1 work log (Word/PDF) exists; Day 2 onward still needs to be written up.

---

8. Remaining plan (3 Nov → 21 Nov)

Rule: each milestone gets its own launch file, a git tag in `~/slam_drone` (to be made a git repo), a screen recording (Win+Alt+R) and a report section. Never break an earlier milestone’s demo.

Milestone 1 — Drone maps the building (3–8 Nov)

1. **Set up the project repo:** `~/slam_drone` as a ROS 2 workspace/package (`git init`), holding nodes, launch files, params and a copy of the model, world and airframe.
2. **Simulation clock:** bridge `/clock` (`/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock`) and run **every** ROS node with `use_sim_time:=true` (static TFs, odom node, slam_toolbox, RViz). Scan timestamps are sim time; mixing wall-clock TF will break SLAM.
3. **Odometry → TF node** (Python, rclpy): subscribe to PX4’s odometry (`/fmu/out/vehicle_odometry…`, best-effort QoS), convert NED/FRD → ENU/FLU, publish the TF `odom → base_link` and `nav_msgs/Odometry` on `/odom`. (In M1 this is PX4’s own estimate; M2 replaces it.)
4. **slam_toolbox** (`online_async`): `scan_topic /scan`, `base_frame base_link`, `odom_frame odom`, `map_frame map`, `max_laser_range 12.0`, `resolution 0.05`.
5. **Waypoint offboard node:** following the `px4_ros_com` offboard example, stream `OffboardControlMode` + `TrajectorySetpoint` at 10 Hz, switch to offboard, arm, take off to 1.5 m, fly the corridor loop (Section 3.3), land. Check exact `/fmu/in/...` names first.
6. **`m1.launch.py`** starting the bridges, TFs, odom node, slam_toolbox and RViz.

- **Demo:** the drone flies the loop while a 2D floor plan draws itself live in RViz, with loop closure snapping it into place. Save the map (`map_saver_cli` or slam_toolbox serialize).
- **Fallback:** fly with teleop/commander instead of the waypoint node.

Milestone 2 — GPS-denied + autonomous (9–15 Nov)

2a GPS-denied (do first):

- Laser odometry (`rf2o_laser_odometry`, built from source for Jazzy) publishes `odom → base_link`. Turn off the M1 odom-TF node so there’s only one publisher of that transform.
- A node converts that odometry ENU → NED and publishes `/fmu/in/vehicle_visual_odometry`.
- PX4 EKF2: GPS fusion off (`EKF2_GPS_CTRL 0`), external vision on (`EKF2_EV_CTRL` horizontal position + velocity + yaw),

height from barometer. Add these to the airframe file.
- **Demo:** the M1 mapping flight repeated with GPS disabled.

2b Autonomous exploration:

- Nav2 (2D, fixed altitude) + `m-explore-ros2` (`explore_lite`, built from source) + a `cmd_vel` → PX4 offboard velocity bridge node (body FLU Twist → NED velocity setpoints, altitude held at 1.5 m).
- **Demo:** press start and the drone maps the whole building by itself.
- **Fallback:** a custom frontier node that picks the nearest unexplored frontier on the occupancy grid and sends PX4 position setpoints directly (no Nav2).

Milestone 3 — 3D map, search-and-rescue, polish (16–21 Nov)

Optional bonuses, in this order:

1. **3D map from the 3D LiDAR:** accumulate `/lidar_3d/points` in the `map` frame (or OctoMap) to get a colour-by-height 3D model of the building.
2. **Victim detection** (cameras are unavailable): put human-sized models in the world and detect them from 3D LiDAR clusters of person-like size, marked as pins on the map. Alternative: retry the camera fix (newer Mesa) with a low-rate camera for YOLO.
3. **Final demo video, report and presentation.**

Guaranteed demo at each point

If stuck in	Can still present
M1	Drone in a building with live 2D/3D LiDAR in ROS 2 (M0 ✅)
M2	Drone mapping the building with SLAM
M3	GPS-denied autonomous mapping drone
All done	Autonomous search-and-rescue mapping drone

9. Work split

- **Kishan:** PX4 side (offboard nodes, odometry into EKF2, flight tuning).
- **Yathivandith:** perception (world, sensors, SLAM, 3D map, victim detection).
- **Both:** Nav2 + exploration integration in M2.